Department of Computer Science and Enginering
University of Puerto Rico
Mayagüez Campus

CIIC 5015 – Introduction to AI Spring 2026
Project I –Search Algorithms
Date: March 14, 2026 11:59 PM

Objectives

1. Understand the design, implementation and use of basic search algorithms
2. Understand the implementation and use of the search in a controlled environment.
3. Use an environment of the game Pacman to illustrate the implementation and use of search methods.

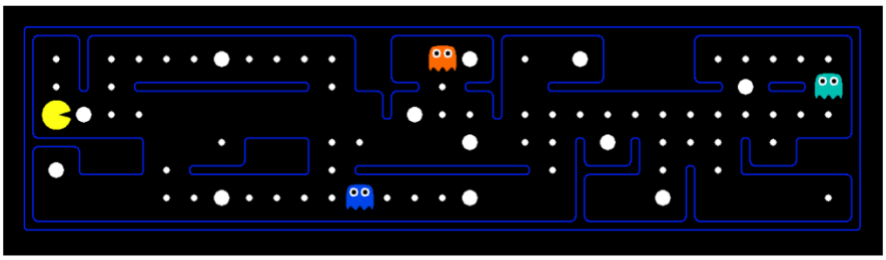
Instructions Deliverables: You will need to provide your solution in moodle

1. You may submit your work in teams of three; however, each team member must understand and be able to run the solution for all tasks.
2. Submit the search.py file containing the solutions to Tasks 1, 2, 3, and 4.
3. Submit a screenshot of the command window showing the solution running.
4. **PROJECT DUE DATE:** 11:59 PM – March 14, 2026.

Introduction

Description of State Spaces and Search Problems, Fundamentally, a search problem is solved by first considering the start state, then exploring the state space using the action and transition and cost methods, iteratively computing children of various states until we arrive at a goal state, at which point we will have determined a path from the start state to the goal state (typically called a **plan**). The order in which states are considered is determined using a predetermined **strategy**. A **world state** contains all information about a given state, whereas a **search state** contains only the information about the world that's necessary for planning (primarily for space efficiency reasons).

The game of Pacman: Pacman must navigate a maze and eat all the (small) food pellets in the maze without being eaten by the malicious patrolling ghosts. If Pacman eats one of the (large) power pellets, he becomes ghost-immune for a set period of time and gains the ability to eat ghosts for points.



First consider a scenario where the maze contains only Pacman and food pellets. We can pose two distinct search problems in this scenario: pathing and eat-all-dots. Pathing attempts to solve the problem of getting from position (x1,y1)(x1,y1) to position (x2,y2)(x2,y2) in the maze optimally, while eat all dots attempts to solve the problem of consuming all food pellets in the maze in the shortest time possible. Below, the states, actions, transition model, and goal test for both problems are listed:

Pathing

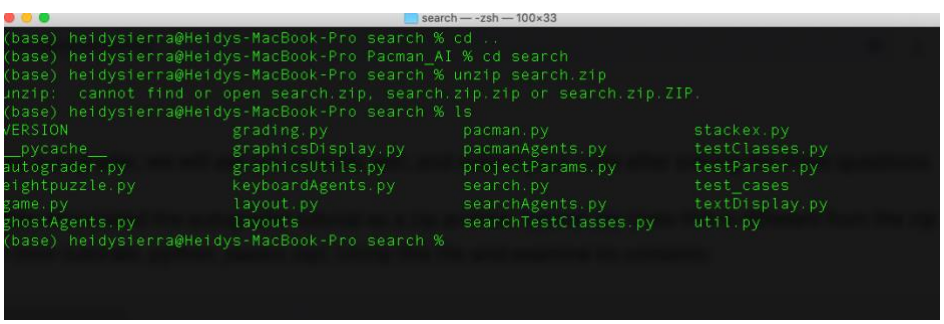
- States: (x,y) locations
- Actions: North, South, East, West
- Transition model (getting the next state): Update location only
- Goal test: Is (x,y)=END?

Eat-all-dots

- States: {(x,y) location, dot booleans}
- Actions: North, South, East, West
- Transition model (getting the next state): Update location and booleans
- Goal test: Are all dot booleans false?

Framework

Go to the terminal and unzip the code and examine the contents



You can play the game by python pacman.py

Pacman lives in a shiny blue world of twisting corridors and tasty round treats. Navigating this world efficiently will be Pacman's first step in mastering his domain.

- The simplest agent in searchAgents.py is called the GoWestAgent, which always goes West (a trivial reflex agent). This agent can occasionally win:
  python pacman.py **--layout** testMaze **--pacman** GoWestAgent
- But, things get ugly for this agent when turning is required:
  python pacman.py **--layout** tinyMaze **--pacman** GoWestAgent
- If Pacman gets stuck, you can exit the game by typing CTRL-c into your terminal.

Project Tasks (there are 3 tasks)

In this project, you will need to implement functions for three search algorithms discussed in class: DFS, BFS, UCS and A*. You will enter the code in the file search.py such as your Pacman agent will find paths through his maze world, both to reach a particular location and to collect food efficiently. You will build general search algorithms and apply them to Pacman scenarios.

The input of each function is a *SearchProblem object* that defines the start state, goal test, successor function, and cost function for a search problem. The input represents a graph and use the following methods:

problem.getStartState()
problem.isGoalState(state)
problem.getSuccessors(state)
problem.getCostOfActions(actions)

Important note: Make sure to use the Stack, Queue and PriorityQueue data structures provided to you in util.py! These data structure implementations have particular properties which are required for compatibility with the autograder.

Output: All your search functions need to return a list of actions that will lead the agent from the start to the goal.

Watch the video: if you know how to unzip the file and run the tests you can skip this video.

The code for this project consists of several Python files. **Files you'll edit:**

search.py Where all of your search algorithms will reside.

searchAgents.py Where all of your search-based agents will reside.

Files you might want to look at:

pacman.py The main file that runs Pacman games. This file describes a Pacman GameState type, which you use in this project.

game.py The logic behind how the Pacman world works. This file describes several supporting types like AgentState, Agent, Direction, and Grid.

util.py Useful data structures for implementing search algorithms.

An autograder will be used to test your solution: By default, the autograder displays graphics with the `-t` option, but doesn't with the `-q` option.
Hint: If Pacman moves too slowly for you, try the option –frameTime 0.

Task 1 (0%): Finding a Fixed Food Dot using Depth First Search (LIFO)

Write the code for the function def depthFirstSearch(problem: SearchProblem) -> List[Directions]: in searchAgents.py. To make your algorithm complete, write the graph search version of DFS, which avoids expanding any already visited states.

Test your code: **Test 1:** First, test the SearchAgent:
python pacman.py -l tinyMaze -p SearchAgent -a fn=tinyMazeSearch

Your code should quickly find a solution for:

python pacman.py -l tinyMaze -p SearchAgent
python pacman.py -l mediumMaze -p SearchAgent
python pacman.py -l bigMaze -z .5 -p SearchAgent

The Pacman board will show an overlay of the states explored, and the order in which they were explored (brighter red means earlier exploration).

Hint: For the Stack structure, the solution found by your DFS algorithm for **mediumMaze should have a length of** 130 (provided you push successors onto the fringe in the order provided by getSuccessors; you might get 246 if you push them in the reverse order). Is this a least cost solution? If not, think about what depth-first search is doing wrong.

Test 2: Please run the below command to see if your implementation passes all the autograder test cases. The following line allows you test question 1 (q1).
python autograder.py -q q1

Task 2 (40%) Bread First Search (FIFO)

Implement the breadthFirstSearch function in search.py. Again, write a graph search algorithm that avoids expanding any already visited states.

Test your code the same way you did for depth-first search.

python pacman.py -l mediumMaze -p SearchAgent -a fn=bfs
python pacman.py -l bigMaze -p SearchAgent -a fn=bfs -z .5

Test your code: Note: If you've written your search code generically, your code should work equally well for the eight-puzzle search problem without any changes.

python eightpuzzle.py

Test 2. Please run the following command to see if your implementation passes all the autograder test cases: python autograder.py -q q2

Task 3 (20pts) Uniform Cost Search Implement the uniform-cost graph search algorithm in the uniformCostSearch function in search.py. We encourage you to look through util.py for some data structures that may be useful in your implementation. You should now observe successful behavior in all three of the following layouts, where the agents below are all UCS agents that differ only in the cost function they use **(the agents and cost functions are written for you):**

Test 1.

```
python pacman.py -l mediumMaze -p SearchAgent -a fn=ucs
python pacman.py -l mediumDottedMaze -p StayEastSearchAgent
python pacman.py -l mediumScaryMaze -p StayWestSearchAgent
```

Note: You should get very low and very high path costs for the StayEastSearchAgent and StayWestSearchAgent respectively, due to their exponential cost functions (see searchAgents.py for details).

Test 2. Grading: Please run the below command to see if your implementation passes all the autograder test cases: python autograder.py -q q3

Task 4 (40%): A* Implement A* graph search in the empty function aStarSearch in search.py. A* takes a heuristic function as an argument. Heuristics take two arguments: a state in the search problem (the main argument), and the problem itself (for reference information). The nullHeuristic heuristic function in search.py is a trivial example.

You can test your A* implementation on the original problem of finding a path through a maze to a fixed position using the Manhattan distance heuristic (implemented already as manhattanHeuristic in searchAgents.py).

python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=astar,heuristic=manhattanHeuristic

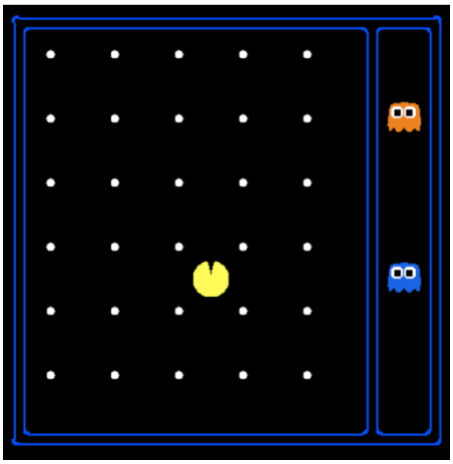
You should see that A* finds the optimal solution slightly faster than uniform cost search (about 549 vs. 620 search nodes expanded in our implementation, but ties in priority may make your numbers differ slightly). What happens on openMaze for the various search strategies?

Test 2: Please run the below command to see if your implementation passes the test cases. python autograder.py -q q4

Additional Helpful Information

Note that for pathing, states contain less information than states for eat-all-dots, because for eat-all-dots we must maintain an array of booleans corresponding to each food pellet and whether or not it's been eaten in the given state. A world state may contain more information still, potentially encoding information about things like total distance traveled by Pacman or all positions visited by Pacman on top of its current (x,y) location and dot booleans.

State Space Size: Solving a search problem bring us to ask about the size of the state space. This is done almost exclusively with the **fundamental counting principle**, which states that if there are n variable objects in a given world which can take on $x_1, x_2, \ldots, x_n$ different values respectively, then the total number of states is $x_1 \cdot x_2 \cdot \ldots \cdot x_n$.



Let's say that the variable objects and their corresponding number of possibilities are as follows:

- *Pacman positions* - Pacman can be in 120 distinct (x,y) positions, and there is only one Pacman
- *Pacman Direction* - this can be North, South, East, or West, for a total of 4 possibilities
- *Ghost positions* - There are two ghosts, each of which can be in 12 distinct (x,y) positions
- *Food pellet configurations* - There are 30 food pellets, each of which can be eaten or not eaten

Using the fundamental counting principle, there are: 120 positions for Pacman, 4 directions Pacman can be facing, 12x12 ghost configurations (12 for each ghost), and $2 \cdot 2 \cdot \ldots \cdot 2 = 2^{30}$ food pellet configurations (each of 30 food pellets has two possible values - eaten or not eaten). This gives us a total state space size of $120 \text{x} 4 \text{x} 12^2 \text{x} 2^{30}$.

State Space Graphs and Search Trees: Recall that a graph is defined by a set of nodes and a set of edges connecting various pairs of nodes. These edges may also have weights associated with them. A **state space graph** is constructed with states representing nodes, with directed edges existing from a state to its children. These edges represent actions, and any associated weights represent the cost of performing the corresponding action. Typically, state space graphs are much too large to store in memory (even our simple Pacman example from above has $\approx 10^{13}$ possible states, yikes!), but they're good to keep in mind conceptually while solving problems. It's also important to note that in a state space graph, each state is represented exactly once, there's simply no need to represent a state multiple times, and knowing this helps quite a bit when trying to reason about search problems.

Acknowledgement: Part of the content of this document is part of the AI Pacman project from UC- Berkeley.